# System Verification and Validation Plan for FPVR

Team 9, Software Mandems
John Cho
Ibrahim Tariq
Vikram Chandar
Sankalpa Chhetri Dhakal
Arjun Sasidaran

November 3, 2026

# Revision History

| Date | Version | Notes |
|---|---|---|
| Date 1 | 1.0 | Notes |
| Date 2 | 1.1 | Notes |

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

# Contents


**1 Symbols, Abbreviations, and Acronyms** **iv**

**2 General Information** **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 3

**3 Plan** **3**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 4
- 3.3 Design Verification . . . 7
- 3.4 Verification and Validation Plan Verification . . . 8
- 3.5 Implementation Verification . . . 9
- 3.6 Automated Testing and Verification Tools . . . 10
- 3.7 Software Validation . . . 10

**4 System Tests** **10**
- 4.1 Tests for Functional Requirements . . . 11
  - 4.1.1 Area of Testing1 . . . 11
  - 4.1.2 Area of Testing2 . . . 12
- 4.2 Tests for Nonfunctional Requirements . . . 12
  - 4.2.1 Area of Testing1 . . . 12
  - 4.2.2 Area of Testing2 . . . 13
- 4.3 Traceability Between Test Cases and Requirements . . . 13

**5 Unit Test Description** **13**
- 5.1 Unit Testing Scope . . . 14
- 5.2 Tests for Functional Requirements . . . 14
  - 5.2.1 Module 1 . . . 14
  - 5.2.2 Module 2 . . . 15
- 5.3 Tests for Nonfunctional Requirements . . . 15
  - 5.3.1 Module ? . . . 15
  - 5.3.2 Module ? . . . 16
- 5.4 Traceability Between Test Cases and Modules . . . 16

**6 Appendix** **17**
6.1 Symbolic Parameters . . . . . 17
6.2 Usability Survey Questions? . . . . . 17

**7 Appendix — Generative AI Use Disclosure** **18**
7.1 AI Tools Used . . . . . 18
7.2 How and Where Used . . . . . 18
7.3 Verification of AI-Generated Content . . . . . 20

**8 Appendix — Reflection** **23**

# List of Tables

1 FPVR verification and validation responsibilities . . . . . 5
[Remove this section if it isn't needed —SS]

# List of Figures

[Remove this section if it isn't needed —SS]

# 1 Symbols, Abbreviations, and Acronyms

| symbol | description |
|---|---|
| T | Test |

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Cho et al., 2026b) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

# 2 General Information

## 2.1 Summary

FPVR is a virtual reality (VR)-based first-person-view (FPV) drone training simulator designed to help beginners develop and improve their drone piloting skills in a safe and accessible environment. Instead of requiring a physical drone, users can operate a simulated drone using VR controls and practise essential flight manoeuvres without the risks of equipment damage or unsafe flying conditions. It is also meant as a tool for experienced Drone fliers to be able to practice in a safe and controlled environment without the pressure of piloting a very expensive piece of equipment.

The system features free-flight practice, progressively challenging obstacle courses, multiple virtual environments, and checkpoints to create a structured learning experience. Users can track their performance through metrics such as completion time, checkpoint progress, and performance ratings, with potential AI-driven feedback to help identify areas for improvement. FPVR also supports registered accounts, saved performance records, attempt history, and local and global leaderboards, allowing users to monitor their progress and compare results with others. Overall, FPVR aims to make learning FPV drone piloting more accessible, engaging, and cost-effective through an immersive virtual training experience.

## 2.2 Objectives

The primary objective of this Verification and Validation plan is to build confidence in the correctness, reliability, usability, performance, and flight realism of FPVR by verifying and demonstrating that all requirements specified in the Software Requirements Specification (SRS) are satisfied. These qualities are major objectives of the SRS and therefore receive explicit attention throughout verification and validation. Verification and validation efforts will be guided by the Hazard Analysis Document, which will be used to prioritize functionality based on its mission criticality, potential impact, and associated risks. Features that are essential to the safe and reliable op-

eration of the simulator will receive the greatest testing attention to ensure that critical system behaviours are thoroughly evaluated. Due to limitations in project time and resources, validating the long-term effectiveness of VR-based training, and testing compatibility across different VR headsets will remain out of scope. Additionally, testing the correctness of the underlying Unity VR libraries, frameworks, and headset-specific functionality will not be part of the project, as these are external components assumed to function according to their respective specifications. Verification efforts will instead focus on FPVR's implementation and integration of these components.

## 2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

The project will include two Custom Documents: a Usability Report and a Design Thinking Report. The Usability Report will document a systematic and quantitative evaluation of FPVR's usability by having representative users complete predefined tasks within the simulator. The evaluation will establish testing goals, identify specific user tasks, collect feedback through structured questions and surveys, and measure usability outcomes to identify areas for improvement. The Design Thinking Report will document the human-centred design process used to understand the needs and challenges of beginner FPV drone pilots. This will include conducting user interviews, organizing findings through empathy maps, generating and evaluating potential solutions based on feasibility and desirability, and developing two rounds of prototypes to gather user feedback and refine the proposed design. These documents will help ensure that FPVR's design decisions are informed by user needs and that its usability is evaluated through structured user testing.

### 2.4 Relevant Documentation

This VnV plan is derived from two project documents: the Software Requirements Specification (SRS) (Cho et al., 2026b) and the Hazard Analysis (Cho et al., 2026a). Together they define what FPVR is supposed to do and which failures would be unacceptable, which is the information needed to choose tests and interpret results.

The SRS (Cho et al., 2026b) is the statement of the software requirements. It records the goals of FPVR, the functional requirements (for example flight control, checkpoint behaviour, and session flow), and the non-functional requirements (for example performance, usability, and safety-related qualities). System tests in this plan are written against those requirements so that a passing test set is evidence that the implemented product matches the specified behaviour. The SRS also supplies identifiers, assumptions, and environmental constraints that this plan uses for traceability: each system test should map back to one or more requirements.

The Hazard Analysis (Cho et al., 2026a) identifies hazards and risks for FPVR. Through a Failure Mode and Effect Analysis it describes how components can fail, what the effects of those failures would be, and which safety and security requirements mitigate them. Those hazards and the derived safety requirements focus on the cases where incorrect behaviour could injure a user, damage equipment, produce misleading training, or compromise the integrity of the application. Tests inspired by the Hazard Analysis therefore include failure and mitigation scenarios (for example invalid controller input, lost tracking, or unrealistic physics) in addition to the happy-path cases taken from the SRS.

## 3 Plan

Verification will check that FPVR satisfies its specified requirements. Validation will check whether the simulator provides understandable controls, useful practice, and meaningful feedback for its intended users. This section assigns responsibilities and describes reviews of the requirements, design, test plan, and implementation, followed by the testing tools and user validation activities.

The working baseline is the current Software Requirements Specification (SRS), especially S.2, S.6, and the environment rules in E.2–E.5. The Prob-

lem Statement supplies the training goals, and the Hazard Analysis supplies failure cases and safety priorities. Accounts, saved results, and leaderboards are included as required by the SRS. Intelligent feedback is conditional on the feasibility decision; multiplayer and dedicated FPV transmitters remain stretch goals. Unresolved differences between documents will be handled through the review in Section 3.2. All activities below are planned work.

Every in-scope functional and non-functional requirement will have a planned verification method and recorded evidence. Functional system tests belong in Section 4.1; non-functional tests, including the separate beginner interface and experienced-pilot realism studies, belong in Section 4.2. Section 4.3 will link these checks to requirements and applicable Hazard Analysis items. Requirements formally excluded from the academic-year scope will be listed with their rationale.

## 3.1 Verification and Validation Team

Responsibilities follow the project roles in SRS P.1. Table 1 identifies the lead for each area. Every member will write tests for their work and review another member's work. The rotating quality assurance (QA) reviewer will not be the main author of the artifact being reviewed, and each pull request will require review by a different member.

The TA and assigned peer reviewers will be asked to review the SRS, design, and VnV Plan against the course expectations. Potential users and experienced pilots will provide validation feedback; their participation is not yet confirmed. John will coordinate invitations, but the project team remains responsible for completing the reviews and resolving findings.

At weekly team meetings, Ibrahim will review open V&V issues and upcoming milestones. Each finding will record its source—peer, TA, supervisor, instructor, or team—affected requirement or hazard, evidence, priority, one owner, requested correction, and a clear condition for closure in GitHub Issues. Larger findings will be split into tasks that one member can complete. The reviewer will check the correction or retest result before closure.

## 3.2 SRS Verification

The team will inspect the SRS before using it to finalize test expectations and repeat the affected checks whenever a requirement changes. Each area lead

Table 1: FPVR verification and validation responsibilities

| **Team member** | **Responsibility** |
|---|---|
| Ibrahim Tariq | Coordinate V&V milestones and defect priorities. Lead flight-model, gravity, camera, and rendering-performance checks, and the experienced-pilot realism study. |
| John Cho | Lead headset, controller, installation, and interruption-recovery testing. Record hardware configurations and arrange supervisor and external reviews. |
| Sankalpa Chhetri Dhakal | Check every course, obstacle type, and recovery position. Lead the beginner interface study and usability findings; evaluate optional intelligent feedback if included. |
| Vikram Chandar | Lead tests of game states, checkpoint order, crash events, respawn, restart, and timers. Maintain repeatable state-transition fixtures. |
| Arjun Sasidaran | Lead score, rating, review, account, storage, and leaderboard tests, including offline recovery and access control. Check expected results independently. |
| Dr. Douglas Down | Review scope, flight realism, and unresolved acceptance criteria during the existing fortnightly meetings. Provide feedback on the Revision 0 demonstration. |

will prepare the relevant requirements, scenarios, and acceptance criteria for a different member to review. The inspection checklist is:

- **Clarity and testability:** Does each requirement have a unique ID, one clear obligation, and an observable result? Are units, limits, timing boundaries, and expected error behaviour defined?
- **Completeness:** Are normal, boundary, and failure cases covered, including first takeoff, repeated collisions, skipped checkpoints, recovery before the first checkpoint, device disconnection, and failed saves?
- **Consistency:** Do the requirements, scenarios, interfaces, environment invariants, and acceptance criteria describe the same behaviour? Are required features clearly separated from conditional features?
- **Traceability and feasibility:** Can each requirement be traced to a project goal or hazard and a planned check? Can that check be performed with the available hardware, people, and schedule?

The following known differences require explicit resolution:

- **Scope:** The Problem Statement and Hazard Analysis still place accounts and database storage among the stretch goals. SRS S.2 requires accounts, cloud results, and leaderboards. Their security checks must therefore be included, and the older roadmap must be reconciled with this scope.
- **Performance:** Hazard Analysis NFR1 specifies 30 FPS, whereas SRS NFR-PERF-01 proposes a 72-FPS average over 120 seconds in each required standalone environment. Hazard Analysis NFR2 specifies a 50-ms response, whereas SRS NFR-PERF-02 proposes at most 100 ms for at least 95 of 100 input events. These measures and thresholds must be agreed and both documents updated before the affected tests can establish acceptance.
- **Recovery and review:** SRS U-03 uses "Restart" for checkpoint recovery, although FR-CHK-05–07 distinguish Respawn from Restart Level. IF-04 excludes prior-session navigation, while FR-REV-05 requires historical results. U-04 also describes recorded flight replay beyond the result-history requirements. The review will align these passages with the agreed scope.

John will arrange a structured supervisor review using a short agenda and the unresolved decisions. The team will walk through free flight, a course with a crash, and a completed result, then ask whether the proposed flight behaviour, score, and feedback support beginner practice. The supervisor will also be asked to help define the reference drone and acceptable realism limits. Peer feedback will be checked against the same checklist and logged as issues.

SRS proposals PR-FLT-01–02, PR-CTL-01, and SR-01–08 will retain their proposed status until reviewed. Hazard Analysis identifiers such as SR1 will be recorded with their document name so they cannot be confused with SRS SR-01. Unset parameters, such as gravity-test tolerances, will be resolved before the corresponding test is accepted. Each approved change will update the source requirement, affected tests, and traceability entries together. Unresolved criteria will be reported as blocked.

## 3.3 Design Verification

The Module Guide and Module Interface Specification will be reviewed as they are developed, first at Design Revision $-1$ and again before module implementation. These documents will define the modules that realize the logical responsibilities in SRS S.1. The area lead will present interfaces and state transitions to another member, and classmates will be asked to review the design through the scheduled peer review.

The design inspection will check that:

- Every required behaviour has an owning module and a traceable interface. Inputs, outputs, units, valid ranges, and failure results are specified, including the controller-to-flight and result-to-storage boundaries.
- State transitions preserve SRS E-I-02: only the next checkpoint counts, one crash produces one penalty, menu time is excluded, Respawn preserves the attempt, and Restart Level creates a fresh attempt. Reviewers will trace these values through a complete example, including a crash before any checkpoint.
- Loss of tracking or input, invalid course data, failed saves, and unavailable services have defined handling. Online or optional AI failures cannot block local training or the standard review. Account ownership and public leaderboard fields are checked at the service boundary.

- Both deployment modes can support the design. Flight-critical work does not wait for network replies, and tests can supply controlled inputs, elapsed time, and service responses without a person wearing the headset.

Interface gaps and conflicting transitions will be resolved before dependent modules are integrated. Small prototypes will check uncertain hardware or performance assumptions, and their results will inform the design review. The team will record review decisions and update the module-to-requirement mapping after design changes.

Document inspections will use the existing team meetings, with about 30–45 minutes reserved for the changed portions at each review milestone. The author and non-author reviewer will read those portions beforehand and retain the completed checklist, decisions, and linked issues. This keeps non-dynamic reviews focused and provides evidence that each review occurred.

## 3.4 Verification and Validation Plan Verification

The rotating QA reviewer will inspect this plan before submission and after major requirement or design changes. The assigned peer reviewers and TA will be asked to assess its clarity and feasibility during the VnV peer review. The checklist is:

- Every in-scope functional, non-functional, and applicable safety requirement has a test or an explicitly justified inspection. Traceability works in both directions, and proposed or deferred items have visible status. Critical recovery, scoring, and data-protection rules have checks at both the logic and integrated-workflow levels.

- Each test gives a unique ID, requirement links, configuration, initial state, inputs, steps, expected result, pass criterion, and the reason for that expectation. Boundary and failure cases are included, and expected values are derived independently of the implementation being tested.

- A different member can execute the procedure and reach the same pass/fail decision. The reviewer will dry-run a checkpoint/recovery test and a performance procedure when the required build is available.

- Equipment, test data, owners, and time are available. Automated tests are separated from headset and user checks, and no procedure requires exposing a participant to deliberately unsafe visuals or physical conditions.

- Section 4.2 contains two separate usability-related test procedures: beginners assess the interface, and experienced FPV pilots assess realism. Section 6.2 contains the shared usability questionnaire and additional questions for experienced FPV pilots. Section 5 retains the unit-testing strategy while detailed cases await the MIS. The List of Figures and List of Tables remain in the document, as required by the rubric.

Once the relevant automated suites exist, their ability to detect faults will be checked using a small, controlled mutation exercise. On a temporary branch, the team will introduce one fault at a time: allow an out-of-order checkpoint, clear total time on Respawn, remove a crash penalty, change a rating boundary, or accept another account's result submission. Each fault must cause its associated test to fail. Undetected faults will lead to a revised test and a repeat check. All mutations will be removed, and the unchanged production code must pass again.

Ibrahim will check progress against SRS P.3: hardware and basic flight at the proof of concept; courses, recovery, and timing by module implementation; scoring and local review by Revision 0; and required online features and verification evidence by the V&V Report. Missing tests or hardware will be reported as incomplete. A milestone is accepted only when its required checks pass and no unresolved defect prevents safe use, required gameplay, or data protection. Scope reductions require a documented requirements change. Tests will not be weakened solely to make a failing build pass.

## 3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

### 3.6 Automated Testing and Verification Tools

[What tools are you using for automated testing. Likely a unit testing framework and maybe a profiling tool, like ValGrind. Other possible tools include a static analyzer, make, continuous integration tools, test coverage tools, etc. Explain your plans for summarizing code coverage metrics. Linters are another important class of tools. For the programming language you select, you should look at the available linters. There may also be tools that verify that coding standards have been respected, like flake9 for Python. —SS]

[If you have already done this in the development plan, you can point to that document. —SS]

[The details of this section will likely evolve as you get closer to the implementation. —SS]

### 3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

## 4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

## 4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

### 4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

**Title for Test**

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

#### 4.1.2 Area of Testing2

...

### 4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

#### 4.2.1 Area of Testing1

**Title for Test**

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

#### 4.2.2 Area of Testing2

...

### 4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

## 5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

## 5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

## 5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

### 5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

Initial State:

Input:

Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

3. ...

#### 5.2.2 Module 2

...

### 5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

#### 5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

Initial State:

Input:

Output:

How test will be performed:

#### 5.3.2 Module ?

...

### 5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

## References

John Cho, Ibrahim Tariq, Vikram Chandar, Sankalpa Chhetri Dhakal, and Arjun Sasidaran. Hazard analysis for fpvr. https://github.com/SoftwareMandems/FPVR/blob/main/docs/HazardAnalysis/HazardAnalysis.pdf, 2026a.

John Cho, Ibrahim Tariq, Vikram Chandar, Sankalpa Chhetri Dhakal, and Arjun Sasidaran. Software requirements specification for fpvr. https://github.com/SoftwareMandems/FPVR/blob/main/docs/SRS/SRS.pdf, 2026b.

# 6 Appendix

This is where you can place additional information.

## 6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

## 6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

# 7 Appendix — Generative AI Use Disclosure

## 7.1 AI Tools Used

- Claude Opus 5.5, Anthropic
- GPT-6 Astra, OpenAI

## 7.2 How and Where Used

1. **How were the AI tools used?**

   Claude Opus 5.5 and GPT-6 Astra were used as drafting, reviewing, and consistency-checking aids. GPT-6 Astra was used mainly to brainstorm section structure, propose candidate tests and survey questions, and rephrase technical writing so it followed the course template fields (control, initial state, input, expected output, derivation, and how the test will be performed). Claude Opus 5.5 was used mainly to turn the team's existing SRS, Hazard Analysis, Development Plan, and Problem Statement into draft paragraphs, to keep requirement identifiers and module names consistent across sections, and to apply the same test layout repeatedly. Both tools were also used to compare this plan against the template and to flag missing subsections, unmatched requirement identifiers, unused template comments, and wording that treated a planned activity as a completed result.

   The tools did not choose the product scope, the two custom documents, the testing priorities, or the out-of-scope items. Those decisions were made by the team. AI-assisted text was written to match them, and unused suggestions were discarded.

2. **Which parts of the document were affected by AI assistance?**

   - **Summary:** GPT-6 Astra produced a first-pass overview of FPVR as the software under test. Claude then rewrote it against the Problem Statement and SRS so the summary named the actual product functions (VR FPV training, free flight, obstacle courses, checkpoints, performance metrics, accounts, and leaderboards) rather than generic simulator features.

- **Objectives:** GPT-6 Astra suggested candidate VnV qualities (correctness, reliability, usability) and a list of possible exclusions. Claude drafted the in-scope objective as verifying FPVR's own implementation against the SRS, with Hazard Analysis used to rank mission-critical behaviour, and wrote the out-of-scope list (long-term training effectiveness, other VR headsets, and Unity or vendor headset software). Those exclusions were then checked against the Development Plan.
- **Custom Documents:** Claude drafted the descriptions of the Usability Report and the Design Thinking Report after the team confirmed those two custom documents from the Problem Statement. The draft states what each report will contain (task-based usability evaluation with surveys; interviews, empathy maps, ideation, and two prototype rounds) and why they belong in this VnV plan rather than as unrelated extras.
- **Relevant Documentation:** Claude drafted the explanation of how the SRS supplies goals, functional and non-functional requirements, and identifiers for traceability, and how the Hazard Analysis supplies failure modes and safety requirements that add invalid-input, tracking-loss, and unrealistic-physics tests. Claude also drafted the BibTeX entries that point at the team's SRS and Hazard Analysis PDFs; the URLs were supplied by the team.
- **Plan sections (team, SRS verification, design verification, VnV-plan verification, implementation verification, automated tools, and software validation):** Both tools were used to check that these subsections still follow the template (roles rather than a name list.
- **System tests:** GPT-6 Astra was used to brainstorm groupings of functional tests (flight, courses, menus, recovery, accounts) and non-functional tests (performance, usability, safety) from the SRS. Claude was used to keep each test in the template format and to attach requirement identifiers. Happy-path cases were taken from the SRS; failure cases (invalid controller input, lost tracking, crash recovery) were taken from the Hazard Analysis.
- **Traceability:** Claude was used to draft or check mappings from test identifiers to SRS requirement identifiers and from unit tests

to the nine logical modules in SRS section S.1, so that a requirement was not left with no planned verification.

- **Unit Test Description:** Claude drafted the introduction, the in-scope and out-of-scope module lists, the black-box / white-box / mocking strategy, and the named automated tests for Flight and Collision, Modes and Levels, Checkpoint Progress and Recovery, Timing and Evaluation, VR Interface and Input, Post-Flight Review, Accounts and Online Service, Results and Leaderboards, and Intelligent Feedback. GPT-6 Astra was used as a second pass to suggest extra edge cases (for example repeated contact producing one crash event, or no crash before takeoff).
- **Usability survey appendix:** GPT-6 Astra helped brainstorm Likert items and background questions. Claude drafted the participant procedure (approximately 6–9 people from the three SRS user profiles), the task script (control guide, free flight, course and crash recovery, post-flight review), the five-point rating items, qualitative follow-ups, and the analysis plan.
- **Wording and consistency review:** Both tools were used throughout to shorten sentences, to keep terminology aligned (requirement identifiers, module names, "system test" versus "unit test," stretch goals versus required tests), and to remove leftover template comments from sections the team had already filled.
- **This disclosure:** Claude Opus 5.5 drafted this appendix from the course template and from the team's record of which sections were AI-assisted. GPT-6 Astra was not used to invent a list of uses that did not occur.

AI-generated suggestions were used as drafts and review notes, not as final text. No figures or diagrams in this document were generated by AI. Citations were not invented by AI.

## 7.3 Verification of AI-Generated Content

1. **How the team checked AI-generated or AI-assisted material before including it in the document.**

   Every AI-assisted change was reviewed by the author of that section before it was committed, and again through pull-request review by an-

other team member before it was merged. Drafts were compared with their sources rather than accepted because they sounded complete: the summary and custom documents against the Problem Statement, objectives and tooling against the Development Plan, system and unit tests against SRS identifiers, and failure and safety tests against the Hazard Analysis. When a tool proposed a new feature, a numeric target, or a change of scope, the team confirmed the decision independently. Several suggestions were rejected for that reason, including treating Unity, OpenXR, or Meta system software as in-scope for FPVR unit tests, treating stretch-goal multiplayer or dedicated transmitters as required tests, and stating headset-wide compatibility testing that the objectives section explicitly excludes. Agreement between Claude and GPT-6 Astra was not treated as verification.

2. **How the team verified factual claims, technical assertions, requirements, calculations, and references.**

    - **Requirements and traceability:** Test cases that name a requirement (for example FR-FLT-03, FR-FLT-05, NFR-USAB-01) were checked against the identifiers defined in the SRS. Tests that name a hazard or safety requirement were checked against the Hazard Analysis. If a draft test cited an identifier that does not exist, or described behaviour that the SRS marks as a stretch goal or likely change, the test was rewritten or marked as applying only if that requirement is in scope.
    - **Technical assertions:** Claims about Unity Test Framework, NUnit, Unity Code Coverage, Edit Mode and Play Mode tests, and the Microsoft C# coding standard were checked against the Development Plan and the vendor documentation for those tools. They are recorded as the team's planned tools, not as evidence that the tests have already been executed.
    - **Calculations and expected outputs:** Expected results in unit tests (for example one crash event from repeated contact, or a decrease in vertical velocity under zero throttle) were derived from the corresponding SRS rules and checked by a team member. Where the flight model is still under review, tests state the direction of change rather than a fabricated numeric trajectory. Score,

rating, and timing oracles will use the formulae in the SRS; they are not taken from an AI-produced example.

- **Usability procedure:** Participant counts, user profiles, and task scripts were checked against Section G.8 of the SRS and the Custom Documents section of this plan so that the survey cannot silently invent a different study. Likert items were reviewed to ensure they measure the intended qualities (control clarity, navigation, comfort, feedback) rather than implementation details the participant cannot observe.
- **References:** AI tools were not used to invent citations. Bibliographic entries for the SRS and Hazard Analysis were written by the team and point at the project repository. The document was compiled with `pdflatex` after substantive edits, and the log was checked for undefined citations and missing labels.
- **Plans versus results:** AI-assisted wording was edited so that this document remains a plan. Coverage targets, frame-rate or latency numbers, and usability scores are treated as quantities to be measured later and reported in the VnV Report, not as verified outcomes of the present draft.

3. **Who is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document.**

   The team is responsible for the accuracy, completeness, consistency, and appropriateness of this Verification and Validation Plan, whether a passage was written by a team member or drafted with AI assistance. Claude Opus 5.5 and GPT-6 Astra can miss requirements, invent identifiers, or describe tests the team cannot run with the time and hardware available. Human review against the SRS, Hazard Analysis, Development Plan, and course template is therefore the check that the plan is feasible, internally consistent, and appropriate for FPVR.

# 8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?